CCDSALG Project: MCO2

Submitted by Jeffrey Eivann Chua,
Kyle Marquez, Darshan Panganiban

References:

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Skiena, S. S. (2020). The algorithm design manual (3rd ed.). Springer.

INTRODUCTION

This project presents an interactive, console-based maze-solving system designed to simulate automated pathfinding. Utilizing a custom-built data structure architecture and an iterative Depth-First Search algorithm, the application dynamically reads maze layouts from text files, animates the traversal process step-by-step in the terminal, and computes real-time performance metrics such as execution time and total steps explored.

DATA STRUCTURE AND DESIGN

In maze simulation we use class "MazeGrid". This class manages an array of Cell objects two dimensions high. Each Cell tracks its (x, y) coordinates as well as its character type and keeps flags for visited, retraced and "solutionPath" to avoid needing separate lookup structure to check status.

DFS itself is driven entirely by custom implementation of a stack. This stack is built from scratch using singly linked list structure to manage active path in memory as the algorithm moves forward and back through the grid.

ALGORITHM & ANALYSIS

The method traverses through mazes by using a Stack to follow paths that have been identified (i. e. , valid unvisited) cells sequentially as they move through them.

When the System encounters an impasse or dead-end, it marks the current cell as having "backed up" and removes it from the Stack allowing the System to "pull-back" and continue with an alternative route at the prior choice point. This results in Time complexity = O(N) and Space complexity = O(N) where N represents the total number of cells within the maze. This max Memory Footprint will occur if all cells in the maze exist on one continuous path forcing each cell to be pushed onto the Stack before any can be removed.

Maze	Size	Type	Cells explored	% of grid	Path length	Found?
Spiral Maze	225	No solution	61	27.10%	N/A	No
Asymmetrical Serpentine Maze	300	Single	132	44.00%	131	Yes
Open Corridors Maze	225	Multiple	76	33.80%	50	Yes
Goal Walled off	225	No Solution	48	21.30%	N/A	No
Start Walled off	225	No Solution	1	0.40%	N/A	No

DISCUSSION

If a solution cannot be found (i. e. , "no solution"), DFS will continue to explore all reachable areas of the graph until it reaches each dead-end; at which point it is safe for the algorithm to terminate, and report that there is no path.

If one unique solution exists, the DFS will stop once it has located the "G" cell, then convert the remainder of the stack into the final solutionPath in order to reconstruct the successful path. If multiple solutions are available, DFS will always follow the first viable solution it encounters, as this is determined by its hard-coded directional priorities, therefore the algorithm does not guarantee that it will find the most efficient or short route.

CONCLUSION & RECOMMENDATION

The application successfully demonstrates core data structures and graph traversal algorithms built entirely from scratch, without relying on external Java collection libraries, and proved robust across a range of maze topologies, from dense branching grids to larger winding paths.

To support shortest-path finding, future versions should integrate Breadth-First Search, potentially utilizing the codebase's existing custom Queue structure.